On the Conservation Laws and Pathway Realizability of Biochemical Reaction Networks

Thanh-An Pham[1] and Tieu-Long Phan[1,2*]

[1]Bioinformatics Group, Department of Computer Science & Interdisciplinary Center for Bioinformatics & School for Embedded and Composite Artificial Intelligence (SECAI), Leipzig University, Härtelstraße 16–18, D-04107, Leipzig, Germany.
[2]Department of Mathematics and Computer Science, University of Southern Denmark, DK-5230, Odense M, Denmark.

*Corresponding author(s). E-mail(s): tieu@bioinf.uni-leipzig.de;
Contributing authors: thanh-an.pham@uni-leipzig.de;

Abstract

Background: Conserved metabolite pools and substrate availability constrain biochemical pathways before kinetic parameters are known. These properties are usually evaluated on incompatible mathematical abstractions: stoichiometric nullspaces omit shared catalytic participants, while graph reachability ignores token multiplicities. Because participants that appear identically on both reaction sides are eliminated under net stoichiometry despite being required for firing, standard reductions can destroy catalytic gating and alter deficiency.

Results: We present the chemical reaction network (CRN) submodule of `SynKit`, an open-source Python toolkit with an optimized C++ core. It builds networks from curated `KEGG` modules, structural `SBML` models, and generative reaction rules into a unified bipartite representation preserving distinct reactant and product incidences. On this representation, `SynKit` computes exact conservation laws, minimal semiflows and siphons, deficiency, and discrete firing certificates that verify whether a reaction multiset can execute from a finite inventory. Twelve reference networks match ground-truth values and agree with `CRNT4SBML` across eight structural invariants, whereas floating-point bases yield corrupted conservation vectors in three cases. Ten `BioModels` entries, up to 504 species and 827 reactions, pass structural round trips; in one of them, cancelling the catalysts listed on both sides of two reactions leaves the net matrix unchanged but lowers the deficiency from 13 to 11. In generative chemistry, formose expansion across six depths yields identical sets of 10 572 species and 11 239 reactions across `SynKit`, `MØD`, and `MolFoundry`. [removed: polyphosphate sentence] In `KEGG` glycolysis, a 15-step pathway certificate realizes two pyruvates from glucose when currency metabolites are omitted, while exhaustive search proves the complete 24-species network is unrealizable without initial cofactor replenishment. In the formose network, the single conservation law reproduces carbon counts for all 37 species, yet precursor pools sharing identical six-carbon budgets diverge in their ability to supply tetrose and hexose targets. **[To do 1.]** Rewrite this Results paragraph last, after notes 5, 7 and 9 are settled; it repeats the floating-point comparison, "exhaustive search proves" and the precursor-pool claim. Draft in the report, section D.

Conclusions: Stoichiometric conservation is necessary but insufficient to guarantee pathway execution. By unifying conservation analysis, deficiency invariants, and Petri-net reachability on a single incidence representation with exact rational arithmetic, `SynKit` makes participant retention, cofactor boundaries, and precursor constraints explicit and testable. Source code, recorded benchmarks, and reproducible analysis workflows accompany the manuscript.

Keywords: Chemical reaction networks, chemical reaction network theory, Petri nets, conserved moieties, stoichiometry, pathway realizability, `SBML`

1 Background

Curated databases such as `KEGG` [1], `Reactome` [2], and `WikiPathways` [3] catalog known metabolic pathways and biochemical reactions, while graph-transformation methods enumerate candidate reactions from molecular seeds and transformation rules [4]. Before assigning kinetic parameters, structural analyses clarify the behavioral constraints imposed by network topology. Specifically, researchers examine which metabolite

pools are conserved by stoichiometric invariants [5], which qualitative dynamical bounds or multistationarity criteria are governed by reaction network structure [6, 7], and whether a candidate pathway is realizable from a finite substrate inventory [8].

Assessing these constraints simultaneously is difficult because each analysis relies on a different mathematical abstraction of the biochemical system. A species graph captures connectivity between metabolites but loses the joint requirement of multiple co-substrates [9]. The net stoichiometric matrix records net changes in species abundance but omits participants that appear identically as reactants and products, such as catalysts or recycled cofactors. For example, a catalyzed reaction $A+C \to B+C$ and the uncatalyzed conversion $A \to B$ yield identical net columns in $\mathbf{S}$, yet only the first requires the catalyst C to proceed. Under mass-action kinetics its rate also depends on C, and the two reactions define different complexes and hence different deficiencies. The risk lies in tools that store reactions as net coefficient maps: `COBRApy` 0.32.1 [10], for example, stores $A + C \to B + C$ as $A \to B$, both when a reaction is built in Python and when it is read from `SBML`. Similarly, a balanced reaction combination or a qualitatively reachable product does not establish that the reactions can proceed in an admissible sequence from a finite inventory [8]. Representation choices during model import can therefore compromise the assumptions of another analysis.

Because of these distinct formulations, existing software packages specialize in separate subsets of analyses: `COBRApy` [10] supports constraint-based flux analysis [11], `CRNT4SBML` [12] evaluates deficiency and potential bistability, `MØD` [4] explores rule-generated reaction spaces, `Tellurium` [13] and `libRoadRunner` [14] simulate kinetic ODEs, and Petri-net methods analyze metabolic invariants [15, 16] and provide pathway realizability certificates [8]. Standard exchange formats such as `SBML` [17] and `COMBINE` [18], alongside converters like `KEGGtranslator` [19], facilitate model transfer. However, converting models between separate formats can inadvertently discard structural features, such as catalysts or boundary species, that specific analyses require.

Conserved pools and substrate sufficiency therefore need to be examined on the same underlying reaction model. In glycolysis [1], a route appears connected from glucose to pyruvate when cofactors are omitted, yet cannot proceed when ATP and redox cofactors are retained with finite initial pools. In a rule-generated formose network [20], every reaction may conserve carbon, yet a proposed pathway remains unrealizable if its reactions require more glycolaldehyde molecules than the initial precursor pool provides. In both cases, conservation laws alone cannot establish whether a reaction sequence is realizable from a specified finite inventory [8].

Here, we present the chemical reaction network (CRN) submodule of `SynKit` [21] to evaluate these constraints on a shared representation. By connecting `KEGG` pathway import, structural `SBML` exchange, and rule-based generation to exact conservation laws, deficiency theory, and Petri-net reachability, the toolkit tests whether reaction sequences can execute without discarding catalysts or cofactors. We validate the implementation across twelve reference networks against `CRNT4SBML`, verify structural round trips on ten `BioModels` entries, and benchmark exact enumeration limits. A curated `BioModels` entry shows how

participant retention changes deficiency, and case studies in glycolysis and formose show how cofactor boundaries and precursor mixtures change pathway realizability, even when net stoichiometry remains unchanged.

2 Implementation

2.1 Network representation and participant incidence

`SynCRN` serves as the core reaction-network data structure in `SynKit`, maintaining species, reactions, transformation rules, and chemical annotations (Fig. 1). It derives bipartite digraphs in `NetworkX` [22], stoichiometric matrices, and Petri nets for downstream structural analysis. The bipartite digraph has species vertices X and reaction vertices $\mathscr{R}$, and each reaction vertex is linked to its reactants and products. It is therefore the incidence representation of a directed hypergraph in which every reaction is a hyperedge, so multi-substrate requirements remain joint hyperedge tails rather than pairwise species edges.

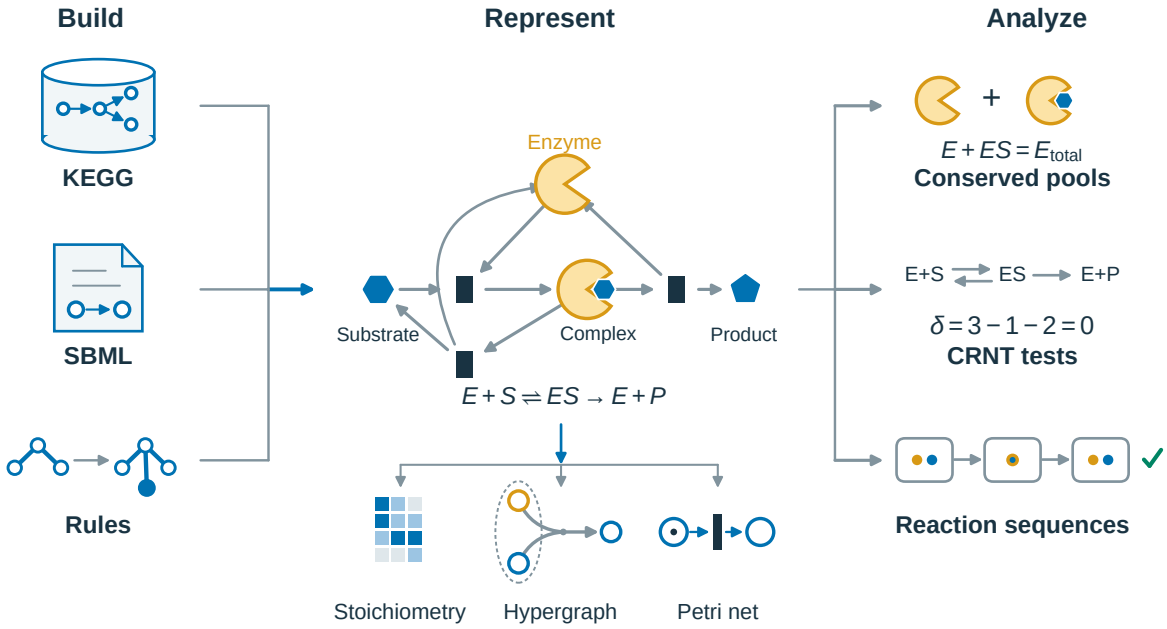


Fig. 1: Overview of `SynKit CRN` workflows across construction, representation, and structural analysis. **Build:** Reaction networks imported from curated databases (`KEGG`), exchange files (`SBML`), or generated by rule expansion are mapped to a unified data structure. **Represent:** An illustrative enzyme mechanism ($E+S \rightleftharpoons ES \rightarrow E+P$) is converted into three complementary views: stoichiometric matrices, directed hypergraphs, and Petri nets, ensuring that multi-reactant requirements (E and S) are captured as joint inputs. **Analyze:** These structural representations enable exact conservation analysis (identifying the conserved pool $E + ES$), chemical reaction network deficiency calculation ($\delta = 0$), and discrete pathway realizability through verified firing sequences. Blue circles denote metabolites, orange circles denote enzymes, and dark bars represent reactions.

For reaction r, reactant and product multisets have coefficients s^-_{xr} and s^+_{xr}, defining separate reactant and product (pre/post) incidence matrices $\mathbf{S}^- = (s^-_{xr})$ and $\mathbf{S}^+ = (s^+_{xr})$ alongside the net stoichiometry $\mathbf{S} = \mathbf{S}^+ - \mathbf{S}^-$. Retaining participants present on both sides in both matrices preserves catalytic and cofactor requirements. Their net coefficients cancel in $\mathbf{S}$, but they still govern reaction enabling and define

distinct reaction complexes (Fig. 2a,b). In Petri-net terms, such shared species form self-loops, and the net incidence matrix alone cannot determine whether transitions are enabled [16]. Construction does not establish elemental balance automatically, so chemical interpretation requires suitable input equations and annotations. The conditions under which a reaction network represents genuine chemical transformations are themselves a subject of study [23].

Reaction networks can be constructed from curated databases, generative chemistry, or standard model files. The `Query` module parses reaction equations from `KEGG`, `Construct` expands networks from chemical seeds using reaction rules, and `IO` imports models from `SBML`. During construction, reversible reactions can be partitioned into directed steps. Users can also choose whether to retain cofactors (such as ATP and NADH) and boundary species, which determines the resulting conserved moieties, network deficiency, and discrete pathway realizability.

Some database equations need care before analysis. `KEGG` writes polyphosphate glucokinase (`R02189`) as `C00404 + C00267 <=> C00404 + C00668`, because one identifier stands for both $(\text{Phosphate})_n$ and $(\text{Phosphate})_{n-1}$. Read literally, the polymer looks like a retained catalyst; chemically it donates one phosphate. `SynKit` keeps such equations as written, and we exclude them from the participant-retention examples below.

In `SBML`, a catalyst appears either as a species listed on both sides, which `SynKit`'s reader keeps, or as a modifier, which it does not import. Six of the ten `BioModels` entries used here contain modifiers; adding them to both sides as participants changes the deficiency of four (Table S3). Because modifiers also encode inhibitors and activators, we do not convert them automatically.

2.2 Conserved moiety and stoichiometric analysis

The stoichiometric matrix $\mathbf{S}$ records the net change in every species for each reaction. For concentrations c and reaction rates $v(c)$, the dynamics are written as $\dot{c} = \mathbf{S}v(c)$. A vector y satisfying $y^T\mathbf{S} = 0$ defines a conserved linear quantity $y^T c$, because the reactions can redistribute the participating species without changing this weighted total. `SynKit CRN` computes a rational basis for the left nullspace of $\mathbf{S}$ and converts its vectors to primitive integer representatives. These vectors describe all rational conservation relations, but they need not be non-negative or identify the smallest conserved pools.

Minimal P-semiflows are computed separately by exact Farkas elimination [24]. They are non-zero, non-negative integer vectors in the left kernel of $\mathbf{S}$ with inclusion-minimal supports, normalized by their greatest common divisor. Their supports identify minimal conserved pools of the represented network [5]. A P-semiflow guarantees mathematical conservation, whereas assigning its support to a chemical moiety requires species annotations and biochemical interpretation.

T-semiflows satisfy $\mathbf{S}x = 0$ with $x \geq 0$ and describe balanced combinations of reactions whose total net species change is zero. They are reaction-space relations, not conserved species pools (Fig. 2c). A

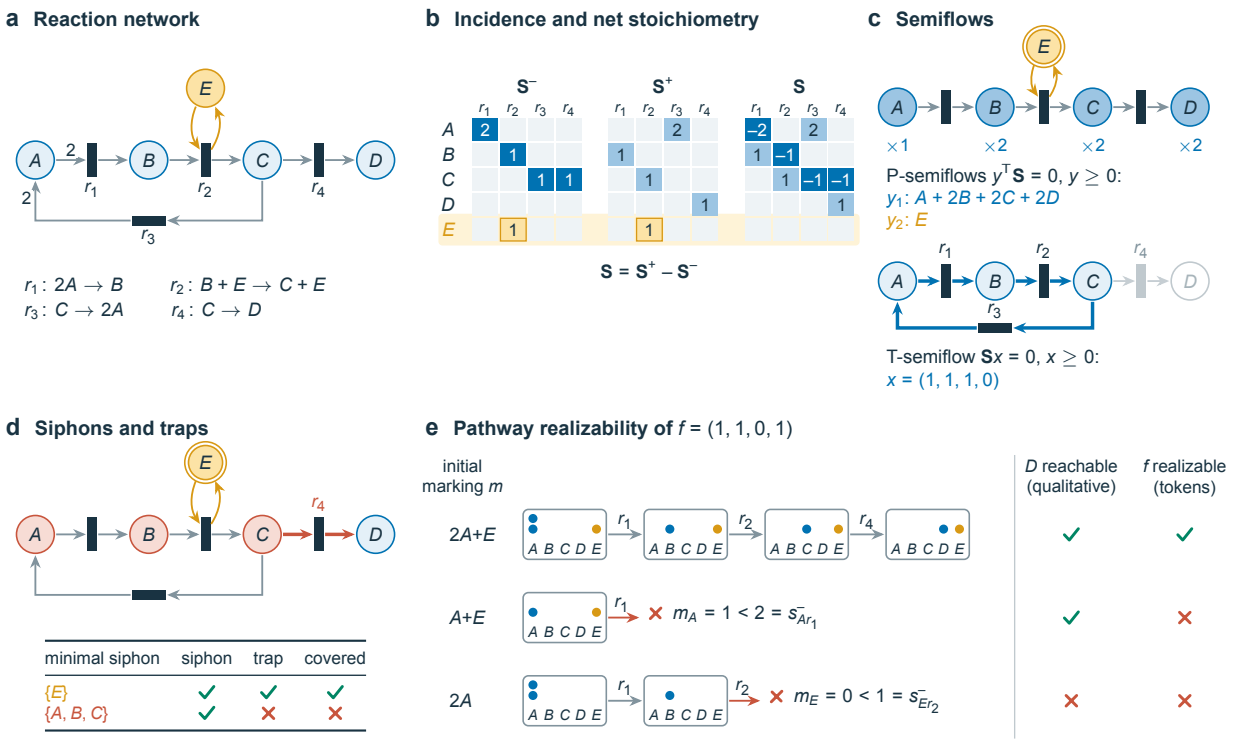


Fig. 2: Structural objects on one toy network, colored as in Fig. 1, with red marking blocked steps or uncovered sets. **(a)** Petri net (circles: species, bars: reactions, arc labels: coefficients other than one). The enzyme E is consumed and returned by r_2. **(b)** Reactant, product, and net incidence matrices. The row of E cancels in $\mathbf{S}$, so the net matrix alone does not show that r_2 requires E. **(c)** P-semiflows (shaded, with weights) are conserved pools. The T-semiflow (heavy arcs) is a balanced reaction combination that fixes no firing order. **(d)** Minimal siphons. $\{E\}$ contains a conserved pool and is covered, whereas $\{A, B, C\}$ drains through r_4 (heavy) and is uncovered. **(e)** Firing from three initial markings (dots: tokens). From $A + E$, qualitative reachability reaches D although no reaction is enabled. Inequalities give the failed condition $m_x \geq s_{\overline{xr}}$.

T-semiflow fixes neither a firing order nor an initial marking, and it does not imply thermodynamic feasibility. All-positive left and right kernel vectors define conservativity and consistency, respectively. To determine positive-kernel feasibility, a linear program proposes a candidate vector that is accepted only after exact rational verification. Feasible networks are certified by an exact all-positive vector, infeasible networks by a rational separating vector, and unresolved solver outcomes remain undecided. Numerical kernel and Jacobian-pattern checks can also characterize approximate subspaces and qualitative matrix patterns, but because floating-point estimates do not isolate integer minimal supports, numerical rounding cannot substitute for exact conservation analysis.

2.3 Deficiency analysis and multistationarity bounds

Complexes are distinct reactant or product multisets. With n complexes, ℓ connected components of the complex graph (linkage classes), and stoichiometric rank s, deficiency is $\delta = n - \ell - s$. A network is weakly reversible when every linkage class is strongly connected. `SynKit CRN` calculates ranks exactly over $\mathbb{Q}$ and reports the deficiency of the network and of each linkage class, weak reversibility, and the terminal strong linkage classes.

For positive mass-action rate constants, a weakly reversible deficiency-zero network has exactly one positive steady state in each positive stoichiometric compatibility class $(c_0 + \operatorname{im} \mathbf{S}) \cap \mathbb{R}^X_{>0}$, and it is locally asymptotically stable relative to that class [25, 6]. A deficiency-zero network that is not weakly reversible has no positive steady state. The Deficiency One Theorem gives at most one positive steady state per positive stoichiometric compatibility class when each linkage-class deficiency is at most one, their sum equals δ, and each linkage class has one terminal strong linkage class, while weak reversibility additionally ensures existence [26]. Failure of these hypotheses leaves the corresponding theorem undecided. Introductions to this body of theory for biologists are available [7]. Criteria that reach beyond deficiency, including injectivity [27], concordance [28], and graph-theoretic multistationarity tests [29], are not implemented here, and a network left undecided by the deficiency theorems is not thereby shown to be multistationary.

2.4 Pathway realizability and siphon analysis

A siphon is a non-empty species set for which every reaction producing a member also consumes a member. Once empty, it cannot refill under Petri-net firing semantics. A trap has the dual property that every reaction consuming a member also produces a member, so a marked trap cannot become empty [16]. `SynKit CRN` enumerates minimal siphons using implication propagation and closure minimization, within the branch-and-bound approach to structural enumeration [30]. Producing a selected species forces selection of a reactant. Selected and excluded species are propagated before branching. Each reported minimal set excludes its supersets from further search, and a maximal-closure deletion test verifies minimality. Traps use the dual incidence (Fig. 2d). This prunes many subsets but does not remove worst-case combinatorial growth.

The package checks whether every minimal siphon contains the support of a P-semiflow, and we call such a siphon covered. Under the boundedness and kinetic assumptions of the corresponding persistence result, this provides a sufficient persistence certificate for positive initial conditions [31]. An uncovered siphon means the test is insufficient, and does not prove extinction or identify a unique nutrient supplementation strategy. The returned details expose these uncovered sets.

A marking m specifies the non-negative integer inventory of each species. Reaction r is enabled precisely when $m_x \geq s^-_{xr}$ for every species, and firing gives $m' = m + \mathbf{S}_{\cdot r}$. Qualitative reachability instead accumulates reachable species sets and ignores consumption and multiplicity. Given a requested flux $f \in \mathbb{N}_0^{|\mathscr{R}|}$, realizability search seeks an enabled sequence firing each reaction exactly f_r times [8]. A positive result includes a certificate consisting of an ordered reaction list that can be checked by updating the inventory after every step. An exhaustive negative result is `unrealizable`, while reaching a state or depth limit is `inconclusive`. [**To do 2.**] `find_siphons` stops at its default limit without warning (`strict_limit=False`); state the limit used, or make the default raise (report C3). Fig. 2e contrasts the two notions on one requested flux. Certificates establish firing in the specified discrete model only.

2.5 Implementation and software comparison

The software employs a hybrid architecture where users interact entirely through a high-level Python interface, while computationally intensive tasks are transparently dispatched to an optimized C++17 library. These native routines compute exact rational nullspaces, CRNT deficiency invariants, minimal semiflows, siphon enumeration, and discrete firing sequences. To eliminate floating-point rounding errors and ensure exact certificates, linear-algebraic and combinatorial routines use arbitrary-precision integer arithmetic via Boost.Multiprecision. For generative chemistry, `SynReactor` applies double-pushout graph transformations with implicit hydrogen counts. Its native C++ controller performs candidate construction, frontier updates, matching, rewriting, and reaction integration from seed molecules. [**To do 3.**] This native DPO controller is not in the `SynKit` 1.6.5 release; release it, or say the timings come from development code (report A4). Following the `MØD` strategy framework [32], each rule application must use at least one newly generated molecule.

To ensure interoperability with the wider modeling community, `SynKit` provides native `SBML` Level 3 Core import and export. The reader and writer preserve species identifiers, compartments, and exact stoichiometries across export and re-import, omitting kinetic rate laws and package extensions. Exported models provide the structural foundation for parameterization in kinetic simulators like `Tellurium` or constraint-based packages like `COBRApy`.

Through this design, `SynKit` occupies a distinct niche in the systems biology software ecosystem (Table 1). Established packages typically specialize in continuous kinetic simulation (`Tellurium`), flux-balance analysis (`COBRApy`), or algebraic deficiency testing (`CRNT4SBML`). While `crnpy` evaluates exact semiflows and `MolFoundry` solves rule-based hyperflows, `SynKit` unifies rule generation, deficiency, minimal siphons, and discrete firing certificates on a shared model. Downstream analyses can then build on exported `SBML` networks. Section 1 of the Supplementary Information and Listing S1 provide complete implementation details and usage examples.

2.6 Validation and benchmark design

We validate the toolkit through analytical benchmarks, runtime scaling experiments, and biochemical case studies. First, structural correctness is evaluated across twelve reference networks spanning enzyme kinetics, phosphorylation cycles, bistable switches, and motifs exhibiting concentration robustness, with shared CRNT invariants independently confirmed by `CRNT4SBML`. Second, computational scaling for exact siphon and conserved-pool enumeration is profiled across synthetic topologies to establish practical tractability limits. Third, `SBML` model exchange and deficiency preservation are tested across eleven curated `BioModels` models up to 504 species and 827 directed reactions.

Two mechanistic case studies demonstrate how structural representation choices dictate pathway feasibility from finite metabolite pools. In `KEGG` glycolysis, we assess how [removed: retaining catalytic polyphosphate and] closing adenine/nicotinamide cofactor cycles governs discrete pathway execution. In rule-generated formose chemistry, we contrast qualitative hypergraph reachability with stoichiometric realizability under

Table 1: Feature comparison of biochemical reaction-network software (✓ supported, (✓) partial or indirect as detailed in footnotes, × not supported). Capabilities are determined from official documentation and source code. `CRNT4SBML` is experimentally evaluated on shared benchmark networks (Table 2), while formose generation is compared with `MØD` and `MolFoundry`, and fixed-route realizability with `MolFoundry` (Section 3.5). The `SynKit` column describes its chemical reaction network (CRN) submodule.

	`SynKit` (this work)	`CRNT4SBML` [12]	`CoNtRol` [33]	`crnpy` [34]	`COBRApy` [10]	`MØD` [4]	`MolFoundry` [35]	`Tellurium` [13, 14]
Network construction and exchange								
`KEGG` pathway import	✓	×	×	×	×	×	×	×
`SBML` import	(✓)[a]	✓	×	✓	✓	×	×	✓
`SBML` export	(✓)[a]	×	×	✓	✓	×	✓	✓
Rule-based network generation	✓	×	×	×	×	✓	✓	×
Structural analysis								
Deficiency and weak reversibility (CRNT)	✓	✓	✓	✓	×	×	×	×
Conservation laws (left-nullspace basis)	✓	✓	×	✓	×	×	×	✓
Minimal semi-positive conservation laws	✓	×	×	✓	×	×	×	×
Elementary flux modes or T-semiflows	✓	×	×	✓	×	×	×	×
Minimal siphons and traps	✓	×	(✓)[b]	×	×	×	×	×
Dynamics and pathway feasibility								
Multistationarity analysis	×	✓	✓	(✓)[c]	×	×	×	×
Pathway realizability (firing sequence)	✓	×	×	×	×	(✓)[d]	✓[f]	×
Constraint-based modeling (e.g., FBA)	×	×	×	×	✓	×	(✓)[f]	×
Kinetic simulation	×	(✓)[e]	×	×	×	×	✓[g]	✓

[a] `SBML` Level 3 Version 2 Core reaction structure only, with kinetic laws and rules not handled. **[To do 4.]** State that an omitted `reversible` attribute (Level 2 default: reversible) is read as irreversible (report C1). Modifiers are now explained in Implementation.
[b] Siphons underlie persistence tests, but minimal-siphon enumeration is not exposed as an output.
[c] `crnpy` supplies Gröbner-basis, elementary-mode, and ACR machinery rather than bistability search.
[d] `MØD` evaluates integer hyperflows rather than discrete firing certificates over a finite marking.
[e] ODE simulation is provided in support of `CRNT4SBML`'s bistability analysis.
[f] `MolFoundry` solves integer hyperflows over derivation graphs (Gurobi or SMT) for firing orders.
[g] Rule-based stochastic simulation with on-demand network expansion.

finite precursor budgets. Each feasible pathway is confirmed by an ordered reaction sequence from available precursors, while infeasibility is proved through exhaustive search.

3 Results and discussion

3.1 Reference-network validation and comparison with `CRNT4SBML`

All twelve reference networks match their analytical and algebraic ground-truth values (Table 2, with sources and validation objectives in Table S1). The benchmark suite spans classic enzyme kinetics (Michaelis–Menten and its reversible counterpart), multisite phosphorylation cycles [36], the bistable Edelstein switch [37], and the Shinar–Feinberg robustness motif [38]. Constructed synthetic cases also verify boundary conditions. Disconnected reversible pairs confirm that independent conserved pools are correctly isolated without conflating disjoint subnetworks, while the open reactor ($0 \to A \to B \to 0$) tests exact handling of inflow and outflow through the zero complex.

To benchmark structural interoperability, `CRNT4SBML` 0.0.15 independently parsed all twelve exported `SBML` models. Across all twelve benchmarks, both tools yielded identical results for all eight shared structural quantities (Table 2), confirming topological consistency across independent implementations. Standard floating-point nullspace methods produce numerical residuals on all closed networks, where scaling and

Table 2: Structural invariants of the twelve reference networks: species, directed reactions, complexes (n), linkage classes (ℓ), stoichiometric rank (s), deficiency (δ), conservation dimension (laws), and weak reversibility (WR). Every network matches its analytical and algebraic ground truth, and `CRNT4SBML` reproduces all eight shared structural quantities.

Network	Species	Reactions	n	ℓ	s	δ	Laws	WR
Reversible isomerization	2	2	2	1	1	0	1	yes
Irreversible isomerization	2	1	2	1	1	0	1	no
Isomerization cycle	3	3	3	1	2	0	1	yes
Disconnected reversible pairs	6	6	6	3	3	0	3	yes
Pairs with stoichiometry	6	6	6	3	3	0	3	yes
Michaelis–Menten	4	3	3	1	2	0	2	no
Reversible Michaelis–Menten	4	4	3	1	2	0	2	yes
Edelstein switch	3	6	5	2	2	1	1	yes
Phosphorylation: one site	6	6	6	2	3	1	3	no
Phosphorylation: two sites	9	12	10	2	6	2	3	no
Shinar–Feinberg motif	2	2	4	2	1	1	1	no
Open reactor	2	3	3	1	2	0	0	yes

rounding fail to recover valid integer conservation vectors on three of the twelve models (Table S1). Exact rational arithmetic guarantees that the conservation identity $y^T\mathbf{S} = 0$ holds across all networks without rounding error. **[To do 5.]** The floating-point comparison uses scale-by-10^3-and-round, a baseline nobody uses; compare against cdd or clingo on every network instead (report B3).

3.2 Computational cost of exact structural enumeration

Enumerating minimal siphons (Fig. 2d) and exact conserved pools (Fig. 2c) identifies species sets that stay empty once depleted and mass conservation laws, but naive search scales exponentially with $2^N - 1$ candidate subsets. To prevent combinatorial explosion, implication pruning propagates stoichiometric constraints along active reactions during search (Fig. 3a). In reversible chains, selecting an initial species forces each adjacent neighbor, isolating the entire chain as the unique minimal siphon without evaluating spurious sub-combinations. Across five repeats per size, pruned search returns identical siphon sets with a 46.6-fold speedup at 21 species (0.63 ms versus 29.25 ms for exhaustive search, Fig. 3b).

The broader benchmark suite evaluates scalability across directed chains, reversible chains, and random sparse bimolecular networks (Fig. S3). All 60 requested configurations complete without numerical failure. On the 401-species reversible chain, minimal siphons (Fig. 2d) take 0.279 s, canonicalization 0.343 s, exact conserved pools (Fig. 2c) 1.02 s, and standalone exact rank 2.15 s (Fig. 3c). The random networks with 49, 99, 197, and 395 species contain 331, 272, 47, and 87 minimal siphons, completing siphon and conserved-pool enumeration in 0.113 s and 0.0789 s at 395 species. Stoichiometric implication pruning and rational nullspace methods thus keep exact structural enumeration computationally practical on networks with hundreds of species.

3.3 Structural `SBML` round-trip validation in `BioModels`

To test real-world structural interoperability and scalability, we evaluated eleven curated `BioModels` [39] entries spanning diverse sizes, including ten reaction networks and one qualitative network control (`SBML-qual`). All ten reaction-bearing models round-trip through the `SBML` reader and writer without loss of stoichiometry, and their ranks and deficiencies agree with an independent exact-rational computation from

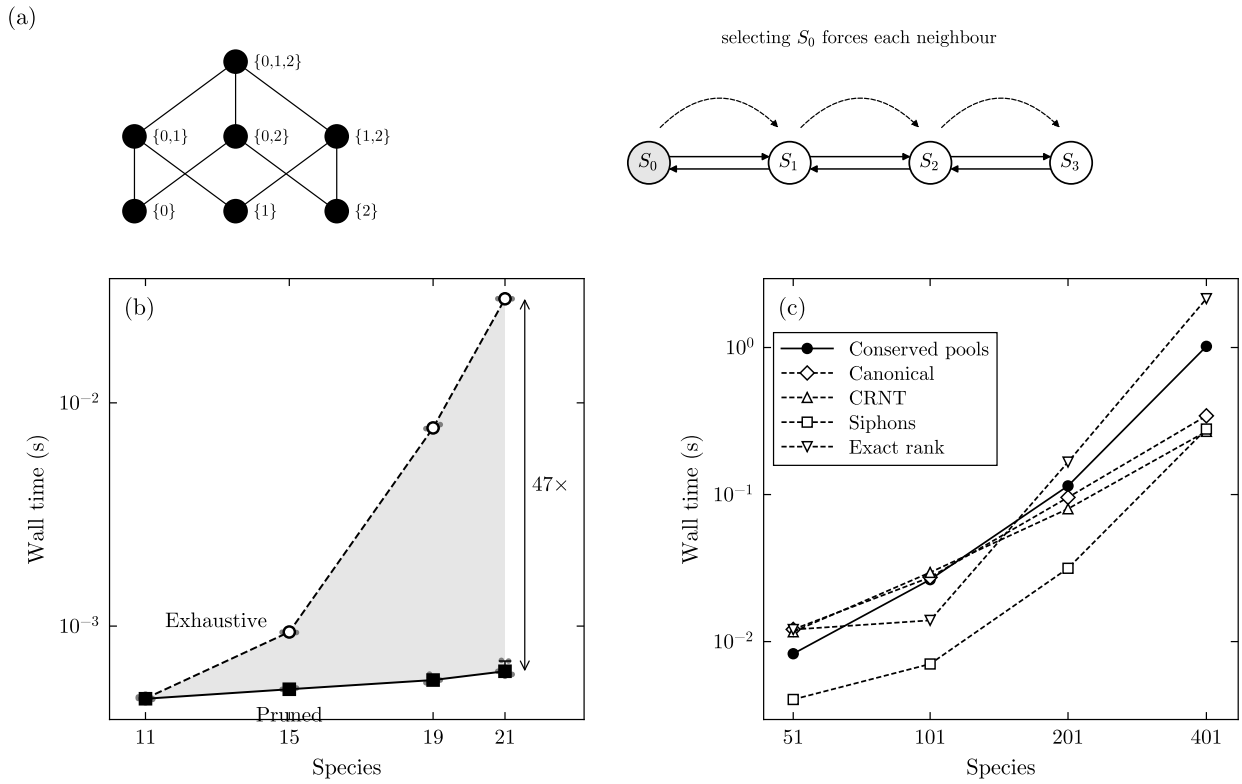


Fig. 3: Search algorithms and runtime scaling on reversible-chain networks. **(a)** Schematic of exhaustive power-set search versus implication pruning, where selecting S_0 forces all chain species into the minimal siphon. **(b)** Minimal-siphon runtimes across five repeats per size (points: individual runs, markers: medians and IQRs, bracket: speedup ratio at 21 species). **(c)** Wall times for five structural analyses across chains up to 401 species.

the SBML reaction lists (Table S2). Even on the largest model (BIOMD0000000255, with 504 species and 827 directed reactions), SynKit identifies 75 minimal siphons in 0.0235 s and 17 exact conserved pools in 2.15 s (Fig. 4a). Testing siphon coverage by conserved pools gives a sufficient condition for structural protection from extinction. In BIOMD0000000255 and the 100-species model BIOMD0000000019 (20 siphons, 19 pools), at least one minimal siphon contains no conserved-pool support, so the sufficient persistence condition [31] cannot certify these models. This does not imply that any species goes extinct; it identifies the species sets that a kinetic analysis should examine first. The qualitative control BIOMD0000000562 is correctly identified as reaction-free for Core structural analysis.

Participant retention matters in curated models as well. In BIOMD0000000404, two dephosphorylation reactions of phosphorylated CheY (Yp) list their catalysts, the phosphatase CheZ (Z) and the receptor complex Tasp_WA, as both reactant and product, for example $\mathrm{Yp} + \mathrm{Z} \rightarrow \mathrm{Y} + \mathrm{Z}$. Cancelling these participants, as net-stoichiometry tools do, leaves $\mathbf{S}$ unchanged but lowers the deficiency from 13 to 11 (Table S3), and it lets dephosphorylation fire without CheZ.

3.4 Conservation and cofactor requirements in glycolysis

Four KEGG [1] pathway modules illustrate structural conservation across metabolic subsystems: glycolysis (M00001), pyruvate oxidation (M00307), the citrate cycle (M00009), and the pentose phosphate pathway

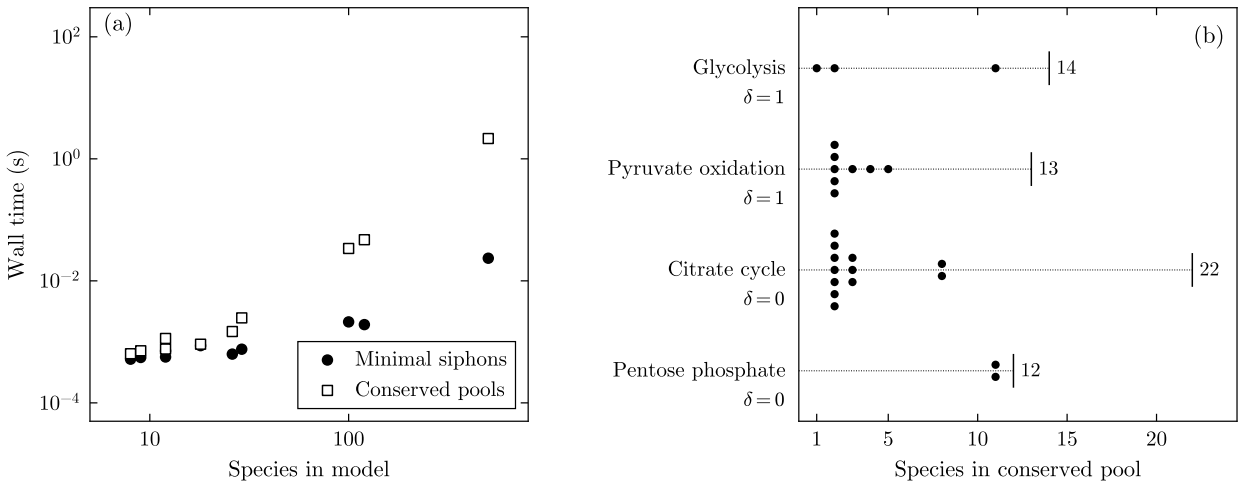


Fig. 4: Enumeration runtimes in `BioModels` and conserved pools in `KEGG` modules. **(a)** Wall time for completed minimal-siphon and exact conserved-pool enumeration in ten reaction-bearing `BioModels` models. All ten models complete both enumerations. **(b)** Minimal non-negative conserved pools in four reduced `KEGG` modules. Each dot is one pool placed at its number of species, pools of equal size are stacked, and the dotted line spans the module species, ending at the module size.

(`M00004`). Their conserved-pool supports appear in Fig. 4b. In the reduced citrate-cycle network, minimal non-negative conservation vectors automatically isolate key metabolic moiety pools, recovering nucleotide pairs, a quinone pair, a ferredoxin pair, coenzyme A, and a lipoyl cofactor pool without manual annotation.

The reduced glycolysis model contains 14 species and 15 directed reactions ($n = 16$, $\ell = 4$, rank $s = 11$, and deficiency $\delta = 1$). Its primary conserved moiety spans eleven carbohydrate intermediates, assigning weight two to hexoses and one to trioses (Fig. 5), capturing exact three-carbon backbone conservation across aldolase cleavage. Glucose and oxidized ferredoxin form uncovered singleton siphons requiring replenishment. [removed: clause on polyphosphate as a covered siphon]

From a single initial glucose molecule, the prescribed glycolytic flux yields a certified 15-step pathway realization terminating in two pyruvates, exploring 30 intermediate markings. Aldolase cleavage generates two triose equivalents that feed the repeated lower payoff phase. Step-by-step certificate replay confirms intermediate availability at every enzyme step and verifies exact invariance of the weighted three-carbon backbone pool ($2 \times$ hexoses $+ 1 \times$ trioses $= 2$). Attempting to execute the lower payoff phase from glucose alone fails because the preparatory intermediates have not yet formed.

[removed: paragraph on net reduction of shared polyphosphate (old l.349)] Restoring currency metabolites (ATP, ADP, NAD^+, NADH, P_i, and PP_i) yields the complete 24-species metabolic network. Supplied solely with glucose, hexokinase cannot fire due to ATP depletion, and exhaustive search certifies that the pathway is unrealizable (Fig. 6). **[To do 7.]** The negative result follows from counting (ATP is consumed before it is produced); give the empty-siphon witness, and note that two initial ATP make the flux realizable (report B1). The reduced model establishes topological feasibility of intermediate conversions, whereas full pathway execution requires balancing cofactor regeneration and energy charge (detailed with inventory controls in Supplementary Section S5).

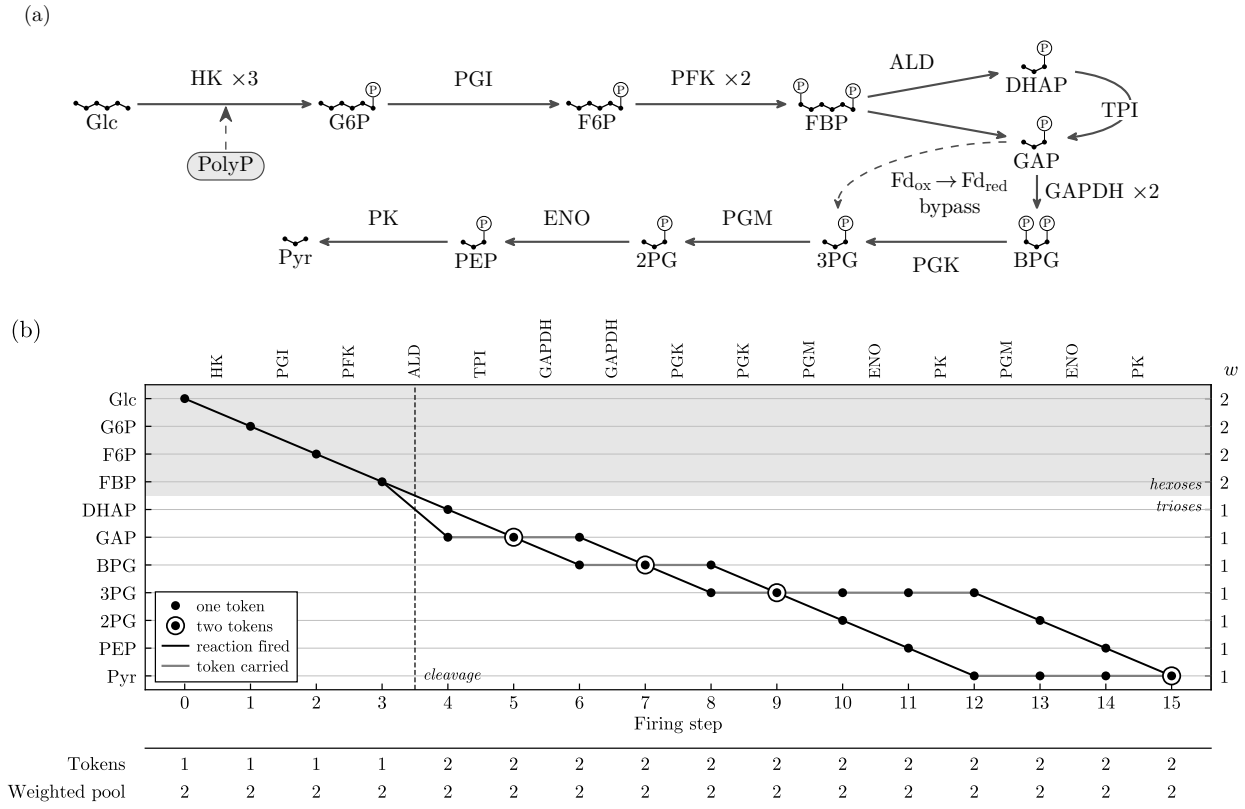


Fig. 5: Conserved pools and pathway realizability in reduced `KEGG` glycolysis (`M00001`). **(a)** Bipartite network retaining polyphosphate and the ferredoxin bypass (dots: carbons, circled P: phosphate). **(b)** Token markings along the certified 15-step conversion of one glucose to two pyruvates (single dot: one token, ringed dot: two tokens). Black lines trace consumed intermediates, gray lines carry unchanged pools, column headers name enzymes, and the dashed line marks aldolase cleavage. Shaded rows indicate hexoses, with pool weights w on the right. Bottom rows trace total carbohydrate tokens and the invariant weighted pool ($2 \times \text{hexoses} + 1 \times \text{trioses} = 2$).

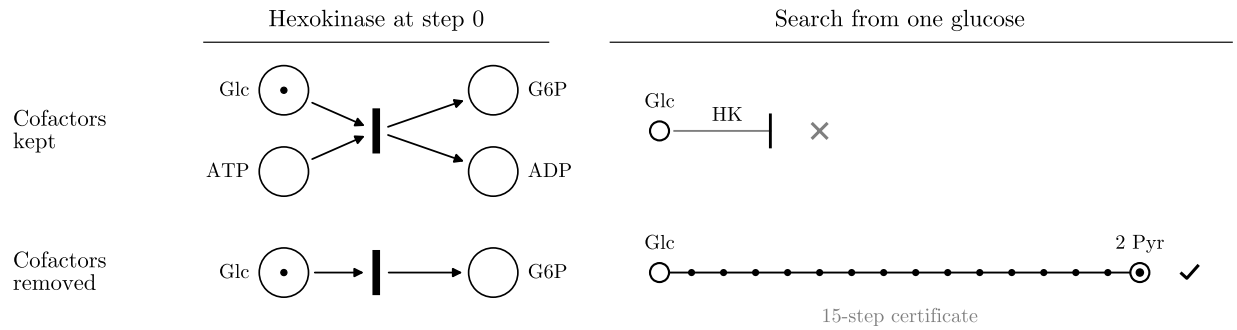


Fig. 6: Pathway realizability of the prescribed glycolytic flux from one glucose under retained versus removed currency metabolites. [removed: panels a and b on polyphosphate; figure redrawn at full width] Left: hexokinase (`R01786`) at step 0. Right: pathway reachability from initial glucose (open circle) through intermediate steps (dots) to two pyruvates (ringed dot).

3.5 Generative chemistry: carbon conservation and precursor requirements in formose

The formose reaction, the classic prebiotic condensation of formaldehyde into diverse sugars [20], provides an open-ended, rule-generated benchmark complementary to curated metabolic networks. Seeded with formaldehyde (C_1) and glycolaldehyde (C_2), four core transformation rules (aldol addition, retro-aldol cleavage, and reversible keto-enol tautomerization through an enediol intermediate) iteratively construct the reaction space. After four expansion rounds, the network comprises 37 species and 44 directed reactions ($n = 54$ complexes, $\ell = 17$ linkage classes, rank $s = 36$, and deficiency $\delta = 1$), reaching decose (C_{10}) species. Median cumulative generation time with the native `SynReactor` DPO controller is 0.0177 s across 20 timed runs after three warm-ups.

Matched expansions in `MØD` 1.0.0.75 [4] and `MolFoundry` 0.1.11 [35] independently validate network generation across six expansion depths (Fig. 7a). At depth six, all three tools generate identical sets of 10 572 species and 11 239 stoichiometric reactions. Median cumulative generation times on an isolated logical CPU are 1.36 s for native `SynReactor` DPO, 1.57 s for `MØD`, and 11.8 s for `MolFoundry` (Fig. 7b and Table S7). These end-to-end API measurements encompass rule parsing, frontier updates, and Python graph construction for `SynKit` without isolating native kernel execution. **[To do 8.]** These DPO timings were produced with code that is not in the cited release; rerun from a tagged release (report A4).

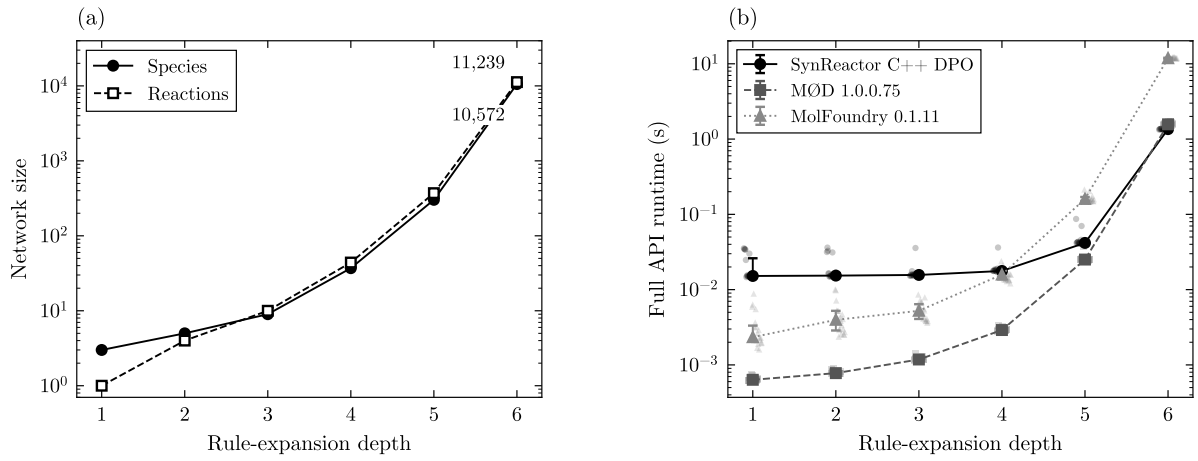


Fig. 7: Formose network growth and full API runtime. **(a)** Species and directed stoichiometric reactions across six rule-expansion depths, with all three tools generating identical sets. **(b)** Full API runtime of native `SynReactor` DPO, `MØD`, and `MolFoundry`. Faint points show 20 timed runs per tool and depth after three warm-ups, large markers show medians, and error bars span the first and third quartiles. Points are offset horizontally for visibility, and times include per-call rule and seed preparation and network construction without imports or engine warm-up. Both panels use logarithmic vertical axes.

Exact nullspace analysis of the four-iteration stoichiometric matrix reveals a one-dimensional left nullspace. Its positive primitive conservation vector agrees identically with the heavy-atom carbon count of every generated molecule, confirming that automated graph transformation preserves carbon conservation across all 37 species (Fig. 8b).

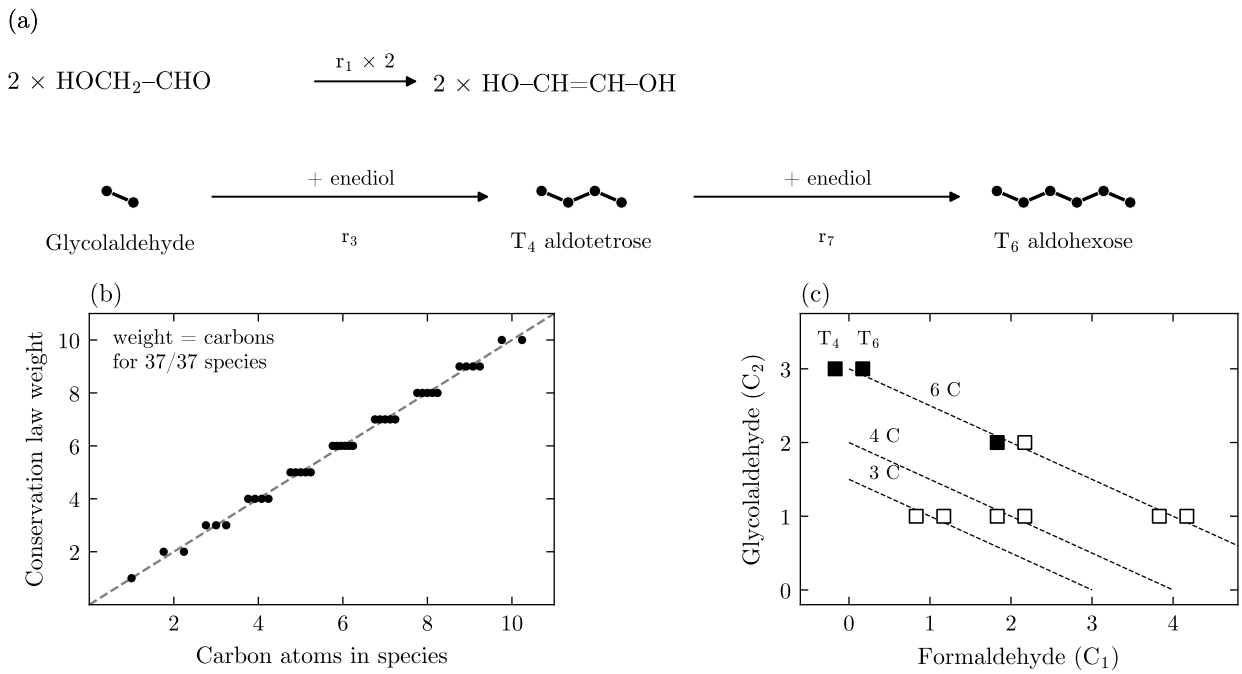


Fig. 8: Carbon conservation and precursor-dependent pathway realizability in the generated formose network. **(a)** Target hexose route requiring three glycolaldehyde precursors via two enolizations (r_1) and two aldol additions (r_3, r_7), each addition consuming one enediol. **(b)** Left-nullspace conservation weights against carbon atom counts across all 37 species (points jittered horizontally, dashed line: $y = x$). **(c)** Five initial inventories by formaldehyde and glycolaldehyde count, with dashed lines joining inventories of equal total carbon. The left and right squares of each inventory refer to the tetrose (T_4) and hexose (T_6) routes, filled when a certificate is verified and open when exhaustive search fails. Inventories with six carbon atoms differ in which routes they can supply.

Qualitative hypergraph search indicates that representative aldotetrose (T_4) and aldohexose (T_6) targets are reachable from the seed pair, but discrete Petri-net realizability differentiates precursor pools sharing identical carbon counts (Fig. 8c). Supplying six carbon atoms as four formaldehydes and one glycolaldehyde (M_3) fails to realize either route. Supplying two formaldehydes and two glycolaldehydes (M_4) supports T_4 synthesis, while three glycolaldehydes without formaldehyde (M_5) realizes both routes. Target synthesis thus hinges on precursor structural assembly and intermediate availability rather than total elemental mass. All ten route–inventory decisions match independent evaluation in `MolFoundry`, and all positive certificates pass independent step-by-step replay (pathway mechanisms and pools detailed in Supplementary Section S6).

[To do 9.] The precursor-pool result follows from the two prescribed routes, which consume no formaldehyde; say so, and add the initiator (siphon) explanation (report B2).

4 Conclusions

The CRN submodule of `SynKit` unites stoichiometric conservation, chemical reaction network theory, and pathway realizability on a bipartite representation preserving distinct reactant and product incidences. Across curated models, metabolic modules and generative chemistries, these analyses demonstrate that net stoichiometric reduction obscures essential physical constraints. Retaining shared participants preserves network deficiency and keeps catalysts such as the phosphatase CheZ required for firing. Similarly, isolating

currency cofactors reveals carbon backbone realizability while showing that full pathway execution requires cofactor replenishment. In generative networks, exact mass conservation is necessary but insufficient for pathway synthesis, where precursor stoichiometric ratios and assembly motifs govern realizability from finite pools. By employing exact rational arithmetic and replayable certificates, `SynKit` makes these modeling assumptions explicit, reproducible, and testable.

These analyses characterize structural enabling and discrete token sufficiency rather than continuous reaction rates, concentrations, thermodynamic driving forces, or *in vivo* regulatory kinetics. Reachability certificates evaluate prescribed reaction multisets rather than unrestricted *de novo* route synthesis, and exact siphon and semiflow enumeration faces combinatorial scaling on dense networks. Structural `SBML` exchange is confined to Core stoichiometric topology, excluding kinetic rate laws and non-core extensions.

Future work will couple discrete firing certificates with estimated reaction free energies ($\Delta_r G$) to filter generated reaction spaces thermodynamically before kinetic simulation. Integrating `SynKit`'s incidence representation with constraint-based flux methods and dynamic ODE simulators will bridge topological feasibility with quantitative metabolic phenotypes, while reducing cross-boundary serialization will scale exploration across complex prebiotic reaction spaces.

Availability and requirements

Project name: `SynKit` (CRN submodule)

Project home page: github.com/TieuLongPhan/SynKit

Software version: `SynKit` 1.6.5 [40] [**To do 10.**] Release a new version with the `SBML` reversible-import fix (report A1) and the native DPO expander (A4), archive it on Zenodo, and cite that version.

Operating system(s): Platform independent (Linux, macOS, Windows)

Programming language: Python, C++17, and C++20

Other requirements: Python $\geq$ 3.10, `NetworkX` $\geq$ 3.3, `NumPy` [41], `SciPy` [42], `RDKit`, Boost headers, CMake [**To do 11.**] Give the exact versions used for the evidence (Python 3.12.12, NumPy 2.4.4, NetworkX 3.6.1, SciPy 1.18.1, RDKit 2026.03.1, Boost 1.90) and add a pinned environment file (report C4).

License: MIT

Any restrictions to use by non-academics: None

List of abbreviations

ACR: Absolute concentration robustness, **CRN**: Chemical reaction network, **CRNT**: Chemical reaction network theory, **DPO**: Double pushout, **EMP**: Embden–Meyerhof–Parnas, **FBA**: Flux balance analysis, **KEGG**: Kyoto Encyclopedia of Genes and Genomes, **SBML**: Systems Biology Markup Language, **SMARTS**: SMILES arbitrary target specification, **TCA**: Tricarboxylic acid, **WR**: Weakly reversible.

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Availability of data and materials

The CRN submodule is distributed with `SynKit` under the MIT License. Source code is available on GitHub at github.com/TieuLongPhan/SynKit [40], and the reproducible analysis scripts are located in the `Experiment/CRN` directory. [**To do 12.**] After the release, rerun the evidence from that tag on the Linux machine, remove the `corrections` note in `analysis/crn-biomodels.json`, and add a Zenodo DOI for the release and scripts. `KEGG` [1] pathways remain subject to `KEGG` terms, and `BioModels` entries are distributed under the CC0 1.0 Public Domain Dedication.

Use of generative AI

Claude Opus (Anthropic) was used for language editing and TikZ figure formatting. The authors reviewed, verified, and take full responsibility for all manuscript content.

Competing interests

The authors declare that they have no competing interests.

Funding

This project has received funding from the European Union's Horizon Europe Doctoral Network programme under the Marie-Skłodowska-Curie grant agreement No 101072930 (TACsy, Training Alliance for Computational systems chemistry). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

Authors' contributions

Author contributions are reported using the CRediT taxonomy. **T.-A.P.**: Software, Validation, Writing (original draft), Writing (review and editing). **T.-L.P.**: Conceptualization, Methodology, Software, Validation, Writing (original draft), Writing (review and editing). Both authors read and approved the final manuscript.

Acknowledgements

Not applicable.

References

[1] Kanehisa M, Furumichi M, Sato Y, Matsuura Y, Ishiguro-Watanabe M (2025) KEGG: biological systems database as a model of the real world. Nucleic Acids Research 53(D1):D672–D677, DOI 10.1093/nar/gkae909

[2] Ragueneau E, Gong C, Sinquin P, Sevilla C, Beavers D, Grentner A, Griss J, Hogue GF, Li NT, Matthews L, et al (2026) The Reactome Knowledgebase 2026. Nucleic Acids Research 54(D1):D673–D681, DOI 10.1093/nar/gkaf1223

[3] Agrawal A, Balcı H, Hanspers K, Coort SL, Martens M, Slenter DN, Ehrhart F, Digles D, Waagmeester A, Wassink I, et al (2024) WikiPathways 2024: next generation pathway database. Nucleic Acids Research 52(D1):D679–D689, DOI 10.1093/nar/gkad960

[4] Andersen JL, Flamm C, Merkle D, Stadler PF (2016) A software package for chemically inspired graph transformation. In: International conference on graph transformation, Springer, Lecture Notes in Computer Science, vol 9761, pp 73–88, DOI 10.1007/978-3-319-40530-8_5

[5] Famili I, Palsson BO (2003) The convex basis of the left null space of the stoichiometric matrix leads to the definition of metabolically meaningful pools. Biophysical Journal 85(1):16–26, DOI 10.1016/S0006-3495(03)74450-6

[6] Feinberg M (1987) Chemical reaction network structure and the stability of complex isothermal reactors—i. the deficiency zero and deficiency one theorems. Chemical Engineering Science 42(10):2229–2268, DOI 10.1016/0009-2509(87)80099-4

[7] Gunawardena J (2003) Chemical reaction network theory for in-silico biologists. https://vcp.med.harvard.edu/papers/crnt.pdf, preprint

[8] Andersen JL, Banke S, Fagerberg R, Flamm C, Merkle D, Stadler PF (2023) On the realisability of chemical pathways. In: Bioinformatics Research and Applications, International Symposium on Bioinformatics Research and Applications, vol 14248, Springer, pp 409–419, DOI 10.1007/978-981-99-7074-2_32

[9] Franzese N, Groce A, Murali T, Ritz A (2019) Hypergraph-based connectivity measures for signaling pathway topologies. PLoS Computational Biology 15(10):e1007,384, DOI 10.1371/journal.pcbi.1007384

[10] Ebrahim A, Lerman JA, Palsson BO, Hyduke DR (2013) COBRApy: COnstraints-Based reconstruction and analysis for Python. BMC Systems Biology 7:74, DOI 10.1186/1752-0509-7-74

[11] Orth JD, Thiele I, Palsson BØ (2010) What is flux balance analysis? Nature Biotechnology 28(3):245–248, DOI 10.1038/nbt.1614

[12] Reyes BC, Otero-Muras I, Shuen MT, Tartakovsky AM, Petyuk VA (2020) CRNT4SBML: a Python package for the detection of bistability in biochemical reaction networks. Bioinformatics 36(12):3922–3924, DOI 10.1093/bioinformatics/btaa241

[13] Choi K, Medley JK, König M, Stocking K, Smith L, Gu S, Sauro HM (2018) Tellurium: An extensible Python-based modeling environment for systems and synthetic biology. BioSystems 171:74–79, DOI 10.1016/j.biosystems.2018.07.006

[14] Welsh C, Xu J, Smith L, König M, Choi K, Sauro HM (2023) libRoadRunner 2.0: a high performance SBML simulation and analysis library. Bioinformatics 39(1):btac770, DOI 10.1093/bioinformatics/btac770

[15] Zevedei-Oancea I, Schuster S (2003) Topological analysis of metabolic networks based on Petri net theory. In Silico Biology 3(3):323–345

[16] Murata T (1989) Petri nets: Properties, analysis and applications. Proceedings of the IEEE 77(4):541–580, DOI 10.1109/5.24143

[17] Hucka M, Finney A, Sauro HM, Bolouri H, Doyle JC, Kitano H, Arkin AP, Bornstein BJ, Bray D, Cornish-Bowden A, et al (2003) The systems biology markup language (SBML): a medium for representation and exchange of biochemical network models. Bioinformatics 19(4):524–531, DOI 10.1093/bioinformatics/btg015

[18] Bergmann FT, Adams R, Moodie S, Cooper J, Glont M, Golebiewski M, Hucka M, Laibe C, Miller AK, Nickerson DP, et al (2014) COMBINE archive and OMEX format: one file to share all information to reproduce a modeling project. BMC Bioinformatics 15(1):369, DOI 10.1186/s12859-014-0369-z

[19] Wrzodek C, Dräger A, Zell A (2011) KEGGtranslator: visualizing and converting the KEGG PATHWAY database to various formats. Bioinformatics 27(16):2314–2315, DOI 10.1093/bioinformatics/btr377

[20] Butlerow A (1861) Bildung einer zuckerartigen Substanz durch Synthese. Justus Liebigs Annalen der Chemie 120(3):295–298, DOI 10.1002/jlac.18611200308

[21] Phan TL, González Laffitte ME, Weinbauer K, Merkle D, Andersen JL, Fagerberg R, Gatter T, Stadler PF (2025) SynKit: A graph-based Python framework for rule-based reaction modeling and analysis. Journal of Chemical Information and Modeling 65(24):13,012–13,019, DOI 10.1021/acs.jcim.5c02123

[22] Hagberg A, Swart PJ, Schult DA (2008) Exploring network structure, dynamics, and function using NetworkX. In: Proceedings of the Python in Science Conference, pp 11–15, DOI 10.25080/TCWV9851

[23] Müller S, Flamm C, Stadler PF (2022) What makes a reaction network “chemical”? Journal of Cheminformatics 14(1):63, DOI 10.1186/s13321-022-00621-8

[24] Colom JM, Silva M (1991) Convex geometry and semiflows in P/T nets: A comparative study of algorithms for computation of minimal p-semiflows. In: Advances in Petri Nets 1990, Lecture Notes in Computer Science, vol 483, Springer, pp 79–112, DOI 10.1007/3-540-53863-1_22

[25] Horn F, Jackson R (1972) General mass action kinetics. Archive for rational mechanics and analysis 47(2):81–116, DOI 10.1007/BF00251225

[26] Feinberg M (1995) The existence and uniqueness of steady states for a class of chemical reaction networks. Archive for Rational Mechanics and Analysis 132(4):311–370, DOI 10.1007/BF00375614

[27] Craciun G, Feinberg M (2005) Multiple equilibria in complex chemical reaction networks: I. the injectivity property. SIAM Journal on Applied Mathematics 65(5):1526–1546, DOI 10.1137/S0036139904440278

[28] Shinar G, Feinberg M (2012) Concordant chemical reaction networks. Mathematical biosciences 240(2):92–113, DOI 10.1016/j.mbs.2012.05.004

[29] Mincheva M, Roussel MR (2007) Graph-theoretic methods for the analysis of chemical and biochemical networks. i. multistability and oscillations in ordinary differential equation models. Journal of Mathematical Biology 55(1):61–86, DOI 10.1007/s00285-007-0099-1

[30] Cordone R, Ferrarini L, Piroddi L (2005) Enumeration algorithms for minimal siphons in Petri nets based on place constraints. IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans 35(6):844–854, DOI 10.1109/TSMCA.2005.853504

[31] Angeli D, De Leenheer P, Sontag ED (2007) A Petri net approach to the study of persistence in chemical reaction networks. Mathematical Biosciences 210(2):598–618, DOI 10.1016/j.mbs.2007.07.003

[32] Andersen JL, Flamm C, Merkle D, Stadler PF (2014) Generic strategies for chemical space exploration. International Journal of Computational Biology and Drug Design 7(2/3):225–258, DOI 10.1504/IJCBDD.2014.061649

[33] Donnell P, Banaji M, Marginean A, Pantea C (2014) CoNtRol: an open source framework for the analysis of chemical reaction networks. Bioinformatics 30(11):1633–1634, DOI 10.1093/bioinformatics/btu063

[34] Tonello E (2016) crnpy: A Python library for the analysis of chemical reaction networks. https://github.com/etonello/crnpy, accessed 7 November 2026

[35] Friedrichs M (2026) MolFoundry: rule-based graph transformation for chemistry, version 0.1.11. https://py.molfoundry.de/docs/, software documentation, accessed 7 November 2026

[36] Wang L, Sontag ED (2008) On the number of steady states in a multiple futile cycle. Journal of Mathematical Biology 57(1):29–52, DOI 10.1007/s00285-007-0145-z

[37] Edelstein BB (1970) Biochemical model with multiple steady states and hysteresis. Journal of Theoretical Biology 29(1):57–62, DOI 10.1016/0022-5193(70)90118-9

[38] Shinar G, Feinberg M (2010) Structural sources of robustness in biochemical reaction networks. Science 327(5971):1389–1391, DOI 10.1126/science.1183372

[39] Malik-Sheriff RS, Glont M, Nguyen TVN, Tiwari K, Roberts MG, Xavier A, Vu MT, Men J, Maire M, Kananathan S, Fairbanks EL, Meyer JP, Arankalle C, Varusai TM, Knight-Schrijver V, Li L, Dueñas-Roca C, Dass G, Keating SM, Park YM, Buso N, Rodriguez N, Hucka M, Hermjakob H

(2020) BioModels—15 years of sharing computational models in life science. Nucleic Acids Research 48(D1):D407–D415, DOI 10.1093/nar/gkz1055

[40] Phan TL, Weinbauer K (2026) SynKit, version 1.6.5. https://github.com/TieuLongPhan/SynKit/releases/tag/v1.6.5, released 30 October 2026

[41] Harris CR, Millman KJ, van der Walt SJ, Gommers R, Virtanen P, Cournapeau D, Wieser E, Taylor J, Berg S, Smith NJ, et al (2020) Array programming with NumPy. Nature 585(7825):357–362, DOI 10.1038/s41586-020-2649-2

[42] Virtanen P, Gommers R, Oliphant TE, Haberland M, Reddy T, Cournapeau D, Burovski E, Peterson P, Weckesser W, Bright J, et al (2020) SciPy 1.0: fundamental algorithms for scientific computing in Python. Nature Methods 17(3):261–272, DOI 10.1038/s41592-019-0686-2